

# **AFYA CENTRO UNIVERSITÁRIO DE MACEIÓ - UNIMA**

**CURSO DE CIÊNCIA DA COMPUTAÇÃO**

## **GERENCIADOR DE TABELA DE SÍMBOLOS**

**Avaliação Prática PBL: Componentes de  
Compiladores**

**DAVI VASCONCELOS DA ROCHA**

Estudante

MACEIÓ  
2026

# RESUMO

**Objetivo:** Implementar um simulador de Tabela de Símbolos focado no gerenciamento de escopos aninhados para a disciplina de Compiladores. **Método:** O projeto foi desenvolvido na linguagem Python, empregando uma estrutura de dados de Pilha de Hash Tables, construída nativamente através de listas e dicionários, onde cada nível da pilha representa um escopo léxico distinto. **Resultados:** O sistema obteve sucesso em demonstrar operações de declaração, busca em múltiplos níveis, sombreamento de variáveis (shadowing) e destruição isolada de escopos locais sem afetar o contexto global. **Conclusões:** A implementação simulou fielmente a análise semântica de um compilador, provando que a abordagem em pilha é eficiente e robusta para resolução de nomes e prevenção de colisões.

**Palavras-chave:** Compiladores. Tabela de Símbolos. Análise Semântica. Python.

# SUMÁRIO

- 1 INTRODUÇÃO
- 2 DESENVOLVIMENTO E RESULTADOS DO ESTUDO
  - 2.1 METODOLOGIA DE IMPLEMENTAÇÃO
  - 2.2 CASOS DE TESTE
- 3 CONSIDERAÇÕES FINAIS
- REFERÊNCIAS

# 1 INTRODUÇÃO

A tabela de símbolos é uma estrutura de dados fundamental na construção de compiladores, atuando principalmente durante as fases de análise semântica e geração de código. Sua responsabilidade é armazenar e gerenciar informações sobre os identificadores (variáveis, funções, classes) encontrados no código-fonte, como seus nomes e tipos.

O gerenciamento de escopo é um dos desafios centrais na implementação dessa estrutura. Em linguagens que suportam escopos aninhados (como blocos de funções ou laços de repetição), uma variável declarada em um escopo interno pode sobrepor ("shadowing") uma variável de mesmo nome em um escopo mais externo, e deve ser destruída assim que o escopo interno é encerrado. Este projeto simula exatamente esse comportamento, utilizando uma abordagem de pilha de tabelas de espalhamento (Hash Tables) para garantir a correta resolução de nomes e a integridade do escopo léxico durante a compilação.

## 2 DESENVOLVIMENTO E RESULTADOS DO ESTUDO

Nesta seção, descrevem-se os procedimentos metodológicos adotados para a construção do simulador, bem como a execução dos testes que validam o comportamento da Tabela de Símbolos.

### 2.1 METODOLOGIA DE IMPLEMENTAÇÃO

O projeto foi desenvolvido na linguagem Python, aproveitando as estruturas de dados nativas para representar os conceitos teóricos exigidos na especificação. A estrutura central (a Pilha de Hash Tables) foi implementada utilizando uma Lista (list), onde cada elemento inserido é um Dicionário (dict). A lista atua como a pilha de escopos, enquanto cada dicionário atua como a Hash Table que mapeia os identificadores para os seus respectivos tipos dentro daquele nível.

A classe principal expõe quatro comportamentos fundamentais:

1. O método de entrar em um escopo adiciona um novo dicionário vazio ao final da lista.
2. O método de sair de um escopo remove o último dicionário inserido, descartando variáveis locais e protegendo o escopo global de exclusão.
3. O método de declaração insere a variável no dicionário do topo da pilha, validando antes se a chave já existe para impedir re-declarações ilegais no mesmo escopo.
4. O método de busca itera sobre a lista de dicionários de forma reversa (do topo da pilha até a base global). Esta iteração reversa garante o funcionamento correto do *shadowing*, priorizando sempre a declaração mais interna e recente.

## 2.2 CASOS DE TESTE

Para validar a implementação, o sistema executa um fluxo de teste automatizado cobrindo os cenários de declaração global, criação de escopo local, sombreamento de variáveis, tentativa de re-declaração inválida e destruição de escopo. A saída do terminal gerada pelo código comprova o funcionamento destas regras semânticas.

Figura 1 – Execução dos casos de teste no terminal

```
PS C:\Users\Davi\Desktop\projeto-compiladores> python tabela_simbolos.py
=== Teste 1: Escopo Global ===
Busca 'x': int
Busca 'y': float

=== Teste 2: Novo Escopo (Local) ===
Busca 'z' (local): string
Busca 'x' (shadowing): boolean
Busca 'y' (herdado): float

=== Teste 3: Conflito no Mesmo Escopo ===
Tentar redeclarar 'z' no mesmo escopo: False

=== Teste 4: Saindo do Escopo ===
Busca 'z' apos sair do escopo: None
Busca 'x' apos sair do escopo: int
```

Fonte: Elaborado pelo autor (2026).

### 3 CONSIDERAÇÕES FINAIS

A execução dos testes demonstra o funcionamento correto da busca em múltiplos níveis e do *shadowing* (onde a busca pela variável local ignorou o tipo da variável global de mesmo nome). O sistema bloqueou com sucesso a concorrência de nomes no mesmo nível, garantindo a consistência das declarações no escopo atual.

A simulação desenvolvida neste projeto enfatiza a relevância de um gerenciamento de memória e escopo seguro na arquitetura de um compilador. A destruição adequada da variável local após a saída do escopo, associada ao retorno do ponteiro de busca para o valor global, reflete de maneira precisa a aplicação do conhecimento teórico de linguagens de programação na resolução de problemas reais de desenvolvimento.

# REFERÊNCIAS

AHO, Alfred V. et al. **Compiladores: princípios, técnicas e ferramentas**. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

PYTHON SOFTWARE FOUNDATION. **The Python Language Reference**. Disponível em: <https://docs.python.org/3/reference/>.